Extension Plan: Diseases to Add, How, and What You'll Need

This document summarizes practical, prioritized recommendations for extending the project with new disease capabilities, what each extension requires (data, models, compute, code changes), and a concrete step-by-step implementation checklist for each suggested disease.

Goals

- Identify high-impact, feasible diseases to add to the project
- For each disease: propose model type, datasets, metrics, preprocessing, training guidance, and integration steps
- Provide a reusable integration checklist so new disease agents can be added consistently
- Summarize non-image (RAG) expansions and governance / validation requirements

---

Prioritized list of candidate diseases/modalities (recommended order)

1. Chest X-ray: Tuberculosis (TB) & Pneumothorax detection
2. Chest X-ray: Multi-label CXR findings (Cardiomegaly, Effusion, Consolidation, Atelectasis)
3. Diabetic Retinopathy (fundus images)
4. Mammography: Breast cancer screening (masses, calcifications)
5. Histopathology: Tumor detection on WSIs (patch classification)
6. Orthopedics: Fracture detection from X-rays
7. Ophthalmology: Age-related Macular Degeneration (OCT or fundus)
8. Dermatology: Expand skin lesion taxonomy beyond melanoma (ISIC classes)
9. Diabetic Foot Ulcer segmentation and classification
10. Endoscopy / Colonoscopy: Polyp detection (object detection)

Rationale: chest X-ray expansions are high-impact, data-rich, and reuse existing code paths. Fundus, mammography and histopathology are clinically valuable but may require modality-specific preprocessing and larger storage. Dermatology expansion is straightforward given existing skin lesion agent.

For each disease: recommended model archetype and datasets

1) Tuberculosis (CXR)

- Task: classification (TB / non-TB) and optional localization
- Model types: DenseNet/EfficientNet/ResNet for classification; Faster R-CNN / YOLO or Grad-CAM for localization
- Public datasets: NIH ChestX-ray14, CheXpert (select TB labels), Shenzhen Hospital TB dataset, Montgomery (public TB sets)
- Preprocessing: resize to 512/1024, histogram equalization/CLAHE optional, lung field cropping (optional)
- Metrics: AUC, sensitivity, specificity, precision-recall, F1
- Integration complexity: low → medium

2) Pneumothorax (CXR)

- Task: segmentation (mask) or classification + localization
- Model types: UNet/nnU-Net for segmentation or Mask R-CNN for instance segmentation
- Datasets: SIIM-ACR Pneumothorax Segmentation (Kaggle)
- Preprocessing: windowing as medical X-ray (normalize), resize, augmentations
- Metrics: Dice / IoU for masks; sensitivity/specificity for presence

3) Diabetic Retinopathy (Fundus)

- Task: multi-class classification (grading 0–4) and lesion detection
- Model types: EfficientNet / ResNet with attention; segmentation for lesions (UNet)
- Datasets: EyePACS (Kaggle DR), Messidor, APTOS
- Preprocessing: color normalization, centered crop, circular mask crop, augmentation (rotation, brightness)
- Metrics: Quadratic weighted kappa, AUC, accuracy

4) Mammography

- Task: detection (masses, calcifications) and classification
- Model types: Faster R-CNN, RetinaNet, and DenseNet for classification; pre-processing for high-res images (tiling)
- Datasets: CBIS-DDSM, DDSM, INbreast
- Preprocessing: high-res tiling, normalization, remove artifacts
- Metrics: sensitivity, specificity, AP/mean AP for detection

5) Histopathology (WSI patches)

- Task: patch-level classification, segmentation of tumor regions
- Model types: ResNet / EfficientNet on patches; MIL (multiple instance learning) for slide-level
- Datasets: CAMELYON16/17, TCGA subsets
- Preprocessing: stain normalization (Reinhard / Macenko), tiling, patch filtering
- Metrics: AUC, slide-level accuracy, IoU for segmentation

6) Fracture Detection (X-ray)

- Task: classification/detection
- Model types: DenseNet / EfficientNet or detection models for localization
- Datasets: MURA (musculoskeletal radiographs), bone fracture datasets
- Preprocessing: grayscale normalization, augmentations
- Metrics: AUC, sensitivity, F1

7) Ophthalmology - AMD (OCT/fundus)

- Task: classification / segmentation depending on modality
- Datasets: public OCT datasets (e.g., Duke OCT), AREDS for fundus
- Preprocessing: modality-specific normalization, cropping

8) Expanded Dermatology (ISIC multi-class)

- Task: multi-class classification (many lesion types) and segmentation
- Model types: EfficientNet + segmentation (UNet) for masks
- Datasets: ISIC (multiple years), HAM10000
- Preprocessing: color normalization, augmentations
- Metrics: per-class recall/precision, macro-F1, confusion matrix

9) Diabetic Foot Ulcer

- Task: segmentation + classification (infection risk)
- Model types: UNet for segmentation with classifier head
- Datasets: DFU datasets (public/academic), may need data collection
- Preprocessing: lesion cropping, color normalization

10) Colon polyp detection

- Task: object detection in video frames or images
- Model types: YOLOv5/YOLOv8, Faster R-CNN
- Datasets: Kvasir-SEG, CVC-ClinicDB
- Preprocessing: resizing, augmentations (flip, blur)

Common data & infrastructure requirements

- Data storage: large disk space for imaging datasets (tens to hundreds of GB depending on modality)
- Compute: NVIDIA GPU (at least one modern GPU; multi-GPU for faster training). Example: single NVIDIA A100/RTX4090 or multi-GPU for large models.
- Annotation tools: CVAT, LabelBox, LabelMe, or specialist tools for medical annotation (ROI, polygon, point)
- Privacy & governance: de-identification of images and patient metadata, data use agreements
- Augmentation / preprocessing pipelines: Albumentations, OpenCV, torchvision transforms
- Training framework: PyTorch preferred (consistent with existing code)
- Experiment tracking: TensorBoard, Weights & Biases, MLflow

Integration checklist (code changes per new disease agent)

1. Create new agent folder:
    - `agents/image_analysis_agent/<disease>_agent/`
    - Files: `inference.py`, `model.py` (optional), `train.py` (optional), `README.md`
2. Add model path in `config.py` under `MedicalCVConfig`:

```
self.<disease>_model_path = "./agents/image_analysis_agent/<di
```

3. Implement inference wrapper `inference.py` with a class exposing `predict(image_path)` returning JSON: `{label, confidence, outputs: {mask_path?, vis_path?}}`.
4. Register agent in `agents/image_analysis_agent/__init__.py`:
    - import class, instantiate in `ImageAnalysisAgent.__init__`, add `analyze_<disease>` method.
5. Update `agents/agent_decision.py` routing rules / system prompt to route images of that modality to the new agent. Optionally update image type classifier to recognize the new modality.
6. Add sample images in `sample_images/<disease>/`.
7. (Optional) Add frontend display logic to show special visual outputs (masks, overlays).

8. Add unit/integration tests for the new agent (lightweight inference tests using a small sample image).
9. Update `PROJECT_CAPABILITIES.md`, `agents/README.md` and `DISEASES_AND_MODELS.md` to document the new capability.
10. Version control & deployment: commit changes, bump version, and add model artifact via release or storage (avoid storing heavy models in git; use release assets or cloud storage).

Training & evaluation recommendations (general)

- Start with transfer learning from ImageNet-pretrained backbones
- Use progressive resizing for stability
- Handle class imbalance using focal loss or class-weighting and oversampling
- Evaluate with cross-validation and report per-class metrics
- Export a deterministic inference pipeline with preprocessing steps encoded
- Save model metadata: training code commit hash, dataset version, hyperparameters, normalization constants

---

Human-in-the-loop & guardrails

- Always return a `confidence` score; define thresholds for `auto-accept`, `flag-for-review`, or `reject`.
- For sensitive diagnoses (e.g., cancer), set `needs_validation=True` by default.
- Log inference events (image id, model version, confidence, clinician decision) for auditability.

Non-image disease support (RAG-driven)

- For textual diseases (guidelines, management), ingest authoritative PDFs into `data/raw/` and run `ingest_rag_data.py`.
- Examples: hypertension guidelines, diabetes care pathways, drug interaction databases.
- The RAG agent can produce source-attributed answers and should be extended by adding curated clinical content.

Regulatory & privacy considerations

- Do not deploy models for clinical use without regulatory approval and clinical validation.
- De-identify PHI and store consent documentation.
- Maintain model versioning and an incident response plan for model errors.

Quick implementation timeline (example for a single disease)

Week 1: dataset curation + preprocessing pipeline Week 2–3: model training (transfer learning) + baseline evaluation Week 4: build inference wrapper + local integration and unit tests Week 5: human validation interface + small clinician study Week 6: finalize docs, CI, and deploy to staging

---

Next steps I can do for you (pick one or more)

- Scaffold a `agents/image_analysis_agent/diabetic_retinopathy_agent/` example (train_stub, inference wrapper, config update)
- Add a training template under `agents/image_analysis_agent/_templates/train_template.py`
- Add unit/integration test skeletons for new agents
- Help prepare a small labeling/annotation pipeline using CVAT or provide sample augmentation scripts

Tell me which option you'd like and I'll implement it next.